Building two small language models from scratch: Telugu and Nepali

Final report for the individual project (Phases 1, 2 and 3)

Jagadish Reddy Kollu (2025201006), Language Models and Agents, Monsoon 2026

Code and all results are on the `phase-1`, `phase-2` and `phase-3` branches

1 What this project was about

I built two small GPT-style language models completely from scratch in PyTorch, one for **Telugu** (my higher-resource language) and one for **Nepali** (my lower-resource language, from the allowed list). I call them Model H and Model L. The two models share code but nothing else: each has its own corpus, its own tokenizer and vocabulary, its own weights, and later its own reasoning dataset. Over the three phases I collected and cleaned the data and trained the tokenizers (Phase 1), implemented the Transformer, pretrained both models on 500M tokens and evaluated them (Phase 2), and then finetuned each one on a synthetic reasoning task and looked at how its attention changed (Phase 3). The table below is the summary; the rest of the report walks through each phase and ends with what I take away from it. The longer per-phase write-ups, with every table and figure, are in `report/phase1`, `phase2` and `phase3` in the repository.

	Model H — Telugu	**Model L — Nepali**
Corpus after cleaning (documents / tokens)	3,139,945 / 674.2M	2,152,701 / 511.3M
Share of training tokens I collected myself (at least 20% required)	26.87%	21.78%
Vocabulary size / characters per token	16,000 / 3.97	16,000 / 4.34
Parameters (11 layers, d=384, 6 heads, tied embeddings)	25,860,480	25,860,480
Pretraining tokens / time on one Kaggle T4	500.0M / 4.5 h	500.0M / 4.6 h
Test perplexity / **test bits per byte**	60.3 / **0.577**	41.7 / **0.468**
chrF++ (t=1.0) / BLEU-4 (t=0.5)	17.03 / 0.41	19.41 / 4.15
Reasoning accuracy (exact match), pretrained → finetuned	0.5% → **26.9%**	0.0% → **25.0%**
Bits per byte after the reasoning finetune	0.959	0.926

Table 1: The main numbers from all three phases.

2 Phase 1: collecting the data and training the tokenizers

Why these two languages. Telugu has plenty of public text (IndicCorpV2, Sangraha, Wikipedia) and its own script, so telling Telugu apart from other languages is trivial. Nepali is on the allowed lower-resource list. It shares the Devanagari script with Hindi, which turned out to be the biggest practical difference between the two pipelines: every downloaded Nepali document had to go through a fastText language-ID check to keep Hindi out, and that check was the slowest step in the whole Nepali pipeline.

Where the text came from. The downloaded part is IndicCorpV2 for both languages, Sangraha for Telugu, the IRIISNEPAL and nepalitext corpora for Nepali, and Wikipedia for both. The spec requires at least 20% of the training tokens to be collected manually, and this was the long pole of Phase 1. I wrote a crawler (`common/scraper.py`) that reads a news site's sitemaps, respects `robots.txt`, rate-limits itself, and keeps its progress in a small SQLite database so it can be stopped and restarted; I ran it for several days on Telugu and Nepali news outlets. The second manual source was public-domain scanned books on archive.org. Searching archive.org by language field found very little; searching by subject and collection instead found about ten times more, which is what made the 20% target reachable. Every record carries a tag saying whether it was crawled or downloaded, and the manual share is counted again in tokens while encoding, so the 26.87% and 21.78% above are measured, not estimated. One thing I only appreciated later: the two manual collections have opposite make-ups. The Telugu one is about 81% news articles, the Nepali one about 59% books. This comes back in Phase 2.

Cleaning. One streaming pass (`common/clean_pipeline.py`): Unicode NFC normalisation, then length and script-ratio filters (at least 80% of letters in the target script), then a few structural checks (mean word length, fraction of duplicated or very short lines), then an exact-duplicate hash, then language ID (confidence of at least 0.60 for Telugu and 0.70 for Nepali), then MinHash near-duplicate detection, and finally a document-level train/validation/test split by hashing the document id with the language seed (1337 for Telugu, 2718 for Nepali). I deduplicate before splitting, and split by whole document, so that no near-duplicate pair can end up with one copy in train and one in test. The 20% rule is enforced by capping how much downloaded text is kept relative to manual text (a ratio of 4.5 for Telugu, 5.2 for Nepali), so in practice it acted as a ceiling on downloaded data rather than a floor on manual data. One limitation

I should state: the MinHash near-duplicate step only ran on the manual halves, because the three overlapping Nepali web crawls did not fit in the 12 GB of memory I had; between those crawls only exact duplicates were removed.

Tokenizers. I trained one SentencePiece BPE model per language with byte fallback on, and swept vocabulary sizes of 8k, 16k, 24k and 32k on 5,000 held-out documents:

	Telugu				Nepali			
vocabulary	8k	**16k**	24k	32k	8k	**16k**	24k	32k
fertility (tokens per word)	2.286	**2.035**	1.919	1.849	1.662	**1.484**	1.410	1.368
characters per token	3.470	**3.898**	4.132	4.289	3.907	**4.375**	4.604	4.747
unknown-token rate	0	0	0	0	0	0	0	0
embedding parameters	3.07M	**6.14M**	9.22M	12.29M	3.07M	**6.14M**	9.22M	12.29M

Table 2: Vocabulary sweep. The chosen size is in bold.

The spec says to choose the size using fertility and the unknown-token rate, but taken literally that rule can only ever pick the largest option: with byte fallback the UNK rate is exactly zero at every size, and fertility just keeps improving as the vocabulary grows. My sweep script therefore picked 32k, and I overrode it to **16,000** for a reason that only shows up in Phase 2: with d=384 and tied embeddings, a 32k vocabulary would spend 12.29M of my roughly 25M parameter budget on the embedding table, half the model, while 16k spends 6.14M and loses only about 10% in fertility. The two vocabularies were built independently; they ended up the same size because the same argument applies to both.

What the tokenizers look like in use. Across the training corpora, 15,872 of the 16,000 Telugu pieces and 15,892 of the Nepali pieces are actually used, the UNK count is zero in both (byte fallback fires instead, for example the Latin x in "Tax" inside a Telugu headline becomes the byte piece `<0x78>`), and the top 1,000 pieces cover 57% of all tokens in both languages, the top 10,000 about 94%. A typical Telugu sentence splits mostly into whole words: రాము హరి కంటే ఎత్తుగా ఉన్నారు becomes ▁రాము ▁హరి ▁కంటే ▁ఎత్తు గా ▁ఉన్నారు; in Nepali the postposition भन्दा ("than") is a piece of its own that attaches to the preceding name. In the end Telugu has 674,176,616 tokens (666.5M for training) and Nepali 511,303,554 (501.3M for training). Both are above the 500M target. The corpora, the tokenized binaries and the tokenizer files are on Google Drive (links in the README); the tokenizer files are also committed.

3 Phase 2: the model, pretraining and evaluation

3.1 The architecture

I wrote the model (`common/model.py`) using only `nn.Linear`, `nn.Embedding`, `nn.LayerNorm` and `nn.Dropout`, with attention written out as explicit matrix multiplications. There is no `nn.Transformer`, no HuggingFace class and no built-in attention function; one of my tests checks that none of these appear in the module. The forward pass goes like this. Token ids of shape (B, T) index a 16000×384 embedding table. A *learned absolute* positional embedding of shape 512×384 is added. I picked learned positions because they are the simplest thing that is clearly my own implementation and they cost only 0.76% of the parameters; the price is a hard cap on sequence length, since there is no row for position 512, so the model cannot process anything longer than 512 tokens. After dropout, the sequence goes through 11 pre-norm blocks of the form $x \leftarrow x + \mathrm{Attn}(\mathrm{LN}(x))$, $x \leftarrow x + \mathrm{FFN}(\mathrm{LN}(x))$, then a final LayerNorm and a linear layer to vocabulary logits.

Inside attention, one fused linear layer produces Q, K and V; each is reshaped from $(B, T, 384)$ to $(B, 6, T, 64)$ so the six heads become a batch dimension and never mix with each other until the end. Scores are $QK^\top/\sqrt{64}$; the scaling matters because without it the dot products have a typical size of about 8, which pushes the softmax into a regime where it outputs almost one-hot weights and its gradient vanishes. A lower-triangular mask adds $-\infty$ to all future positions *before* the softmax, so those positions get exactly zero weight. The heads are concatenated and projected by W_O. I use pre-norm rather than post-norm because with pre-norm the residual path from the loss back to the first layer is a plain sum of identities, which is what lets an 11-layer stack train without a delicate warmup. The feed-forward layer is $384 \to 1536 \to 384$ with GELU and holds about two thirds of each block's parameters. Dropout of 0.1 is applied to the embeddings, the attention weights and both sublayer outputs. The output layer shares its weights with the input embedding, which saves $16000\times384 = 6.14$M parameters, about 24% of the model. The total is **25,860,480** parameters. I chose 11 layers of width 384 rather than something shallower and wider because at 512 tokens attention cost does not depend on the head width while the feed-forward cost grows with the square of the width, so depth buys more computation per parameter. The causal mask is checked empirically by a test that changes token $t{+}1$ and asserts that the logits at position t do not change at all.

3.2 Pretraining

Both models use the same hyperparameters, which are in `telugu/configs/model.yaml` and `nepali/configs/model.yaml` and are also written into every checkpoint: AdamW with a peak learning rate of 6×10^{-4}, 10M tokens of linear warmup then cosine decay to 6×10^{-5}, $\beta = (0.9, 0.95)$, weight decay 0.1 on matrices only, gradient clipping at 1.0, a micro-batch of 32 sequences with 2 accumulation steps (32,768 tokens per optimizer step), fp16 with a GradScaler because the T4 has no bf16, and 15,259 steps for 500,006,912 tokens, one pass over the data. I picked the batch geometry by measuring: 32×2 gave 34.5k tokens/s at 9.7 GB, 16×4 was slower, and 64×1 ran out of memory.

Checkpointing got a lot of attention because free GPU sessions die without warning. Every checkpoint contains the weights, both AdamW moment estimates, the GradScaler state, the step and token count, the full config, and the NumPy, CPU and CUDA random states. The learning rate is a pure function of the token count, so there is no scheduler object to serialise, and files are written to a temporary name and renamed so a crash mid-write cannot corrupt the last good checkpoint. My test for this trains a model straight through, then trains an identical one with a stop and a resume in the middle, and asserts the weights are bit-for-bit identical (with dropout on; an earlier version that only saved the CPU random state passed on CPU and would have silently diverged on GPU). I repeated that check on Kaggle before spending the GPU hours. The Telugu run took 4.50 hours and the Nepali run 4.58 hours on one T4.

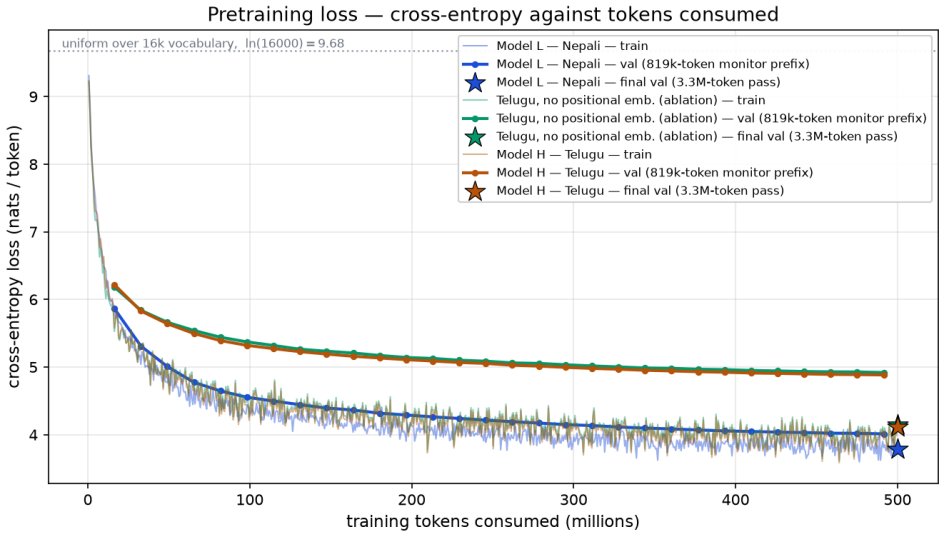


Figure 1: Training loss, the validation monitor computed during training, and the final whole-split evaluation (drawn as a separate marker because it is computed on more data than the monitor).

Both curves are smooth, no loss spikes, and both runs land exactly on the minimum learning rate at the end. Training and validation loss finish within 0.12 nats of each other for Telugu and 0.003 for Nepali, so neither model is overfitting; at 500M tokens for 25.9M parameters they are close to the Chinchilla ratio and simply data-limited. The Telugu validation monitor sits about 0.9 nats above its training loss from the very first evaluation, which is not overfitting (that gap would open over time) but a sign that the head of the Telugu validation split is a harder slice of the corpus than the training mix; the whole-split number is much closer.

3.3 How good are they as language models?

I evaluate on the whole held-out split with non-overlapping windows so the number is a property of the data and not of a random draw.

	validation		**test**	
	Telugu	Nepali	Telugu	Nepali
tokens scored	3,751,936	4,964,352	3,899,392	4,997,120
cross-entropy (nats per token)	4.0637	3.7226	4.0987	3.7306
perplexity	58.19	41.37	60.26	41.70
bits per byte	**0.5680**	**0.4662**	**0.5770**	**0.4675**

Table 3: Language-modelling results of the pretrained models on their own held-out data.

Perplexity is per token, and a Nepali token carries about 12% more text than a Telugu one (11.5 vs 10.3 UTF-8 bytes), so perplexity alone would flatter Nepali. Bits per byte divides the cross-entropy by $\ln 2$ times the bytes per token,

which removes the tokenizer from the comparison. Even on that measure the lower-resource model is better, by 19%. I did not expect that, and I come back to it in the discussion.

Generation. I took 200 prefixes of 64 tokens from the test split, generated 64 tokens with greedy decoding and with temperatures 0.5, 1.0 and 1.5, and compared with the real continuation.

	Telugu				Nepali			
	greedy	t=0.5	t=1.0	t=1.5	greedy	t=0.5	t=1.0	t=1.5
BLEU-4	0.34	**0.41**	0.09	0.03	3.83	**4.15**	3.61	0.86
chrF++	10.2	14.4	**17.0**	16.2	15.3	17.7	**19.4**	16.0
ROUGE-L	0.025	**0.031**	0.021	0.009	0.087	**0.093**	0.087	0.033
4-gram repetition rate	0.65	0.15	0.00	0.00	0.50	0.19	0.00	0.00
Distinct-1 / Distinct-2	.13/.19	.40/.67	.75/.99	.83/1.0	.16/.26	.31/.61	.56/.93	.72/.99

Table 4: Generation metrics under the four decoding settings.

Two implementation details were necessary to get meaningful numbers at all: sacreBLEU's default tokenizer treats a whole run of Telugu or Devanagari as a single token, so I run it with `tokenize="none"` on whitespace-split text, and the standard ROUGE package strips everything non-ASCII, so I implemented ROUGE-L directly as an LCS F-measure. On whether these metrics are informative: honestly, not very. All three reward overlap with the one continuation that happened to follow in the corpus, and there is no reason a good continuation should match it. They also fail unequally. At t=1.0 Nepali beats Telugu by 41$\times$ on BLEU-4 but only 1.14$\times$ on chrF++, while the true gap in bits per byte is 1.23$\times$. Telugu is more strongly agglutinative, so a single morphological change alters a whole word and kills a word-level n-gram match while barely moving character overlap; BLEU on Telugu is measuring Telugu morphology, not the model. chrF++ is the only one of the three I would trust across the two languages. The diversity numbers tell a cleaner story: greedy decoding falls into loops (two thirds of Telugu 4-grams are repeats), t=0.5 is the sweet spot where the overlap and diversity metrics agree, and t=1.5 produces text where nearly every bigram is unique, which at this model size means noise rather than variety.

Reading the samples: both models produce real words and correct grammar within a clause, and stay on topic for four or five words before drifting to something unrelated. For example, a Telugu prefix about a family sleeping at a bus station with an 18-month-old baby was continued at t=0.5 with a sentence that is grammatical but has drifted to "two heroines acted in this film, which was released in 2019 and was a super hit". That four-to-five-word horizon matches the range of the local attention heads described next.

3.4 What the attention heads learned

For each model I ran three held-out passages through it and computed, per head, the entropy of the attention distribution, the average distance it looks back, the mass it puts on position 0 (the "sink" that transformers use as a place to park unused attention) and the mass it puts on the previous token. One correction is needed to make these numbers meaningful: because of the causal mask, a query at position i can only spread over i+1 keys, so raw entropy mostly measures how far into the sequence you are. I normalise entropy by $\ln(i{+}1)$ and skip the first 16 positions.

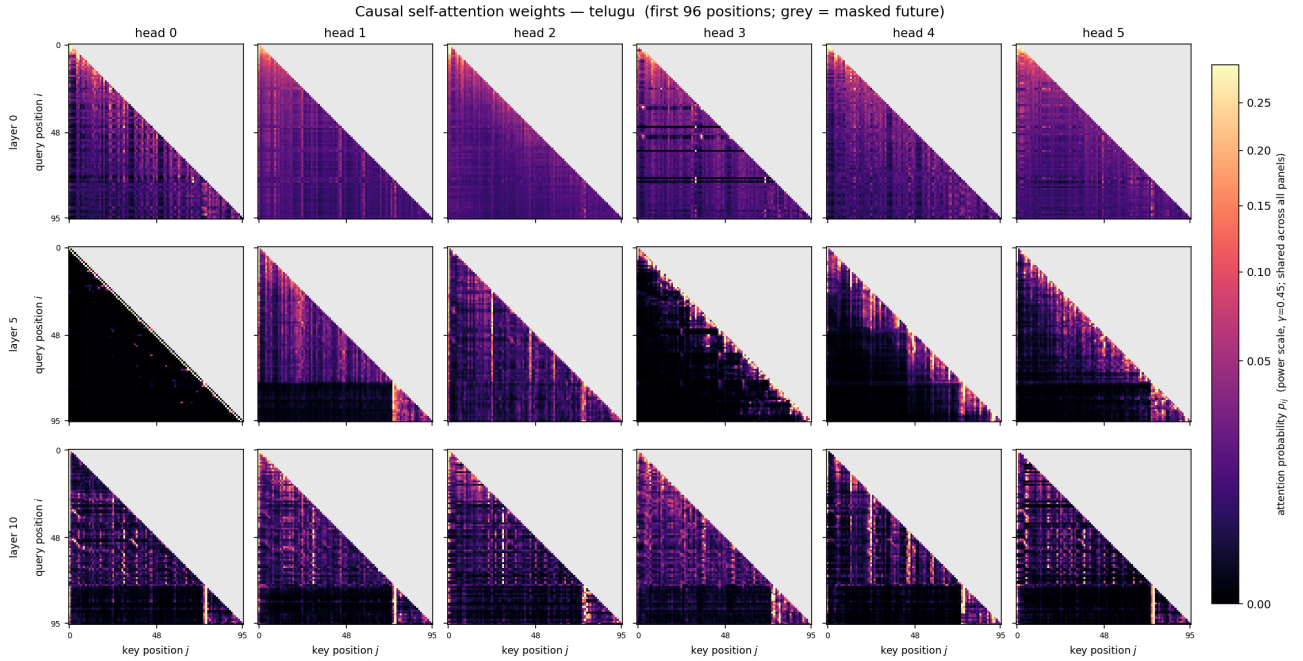


Figure 2: Model H's attention on a held-out Telugu passage, layers 0, 5 and 10, six heads each, with one shared colour scale (grey is the masked future). The Nepali figure in the repository looks almost the same.

	Telugu			Nepali		
layers	norm. entropy	distance (tokens)	previous-token mass	norm. entropy	distance (tokens)	previous-token mass
0–2	0.93–0.83	23–28	0.03–0.07	0.94–0.76	25–29	0.03–0.07
3–5	0.57–0.51	6–10	0.21–0.28	0.59–0.52	6–12	0.21–0.26
6–10	0.49–0.69	15–22	0.04–0.10	0.53–0.71	16–27	0.03–0.10

Table 5: Attention statistics by stage. Head counts: Telugu 29 local, 20 diffuse, 17 content, 0 sink; Nepali 24, 25, 16, 1.

The thing I found most striking is that the two models, trained on unrelated corpora with different seeds, organised themselves the same way. Layers 0–2 attend almost uniformly over everything visible. Layers 3–5 are local: the average look-back drops to about six tokens and the mass on the previous token peaks, and in both models layer 5 head 0 is a pure previous-token head, a single bright sub-diagonal. Layers 6–10 look further back again but selectively, on content rather than position. Both models also learned, with no supervision for it, to stop attending across document boundaries: on passages that cross an `</s><s>` pair, layers 5 and 10 show a clean L-shaped dark region starting exactly at the boundary. Only one head out of 132 across both models is an idle sink head, so the capacity is being used.

3.5 Bonus: what happens without positional embeddings

I retrained Model H with the positional table removed and nothing else changed. My prediction was a loss of a few tenths of a nat. The actual cost was **0.041 nats** (test cross-entropy 4.099 → 4.140, bits per byte 0.577 → 0.583, under 1%), the same on validation and test, and chrF++ did not move (17.03 vs 17.02) while BLEU-4 fell by 42%: character-level fluency is untouched, exact word order gets slightly worse. The reason it costs so little is that the causal mask is itself a position signal, because query t sees exactly $t{+}1$ keys, and a decoder can recover order from that. The attention statistics show where the small cost comes from: the same three-stage organisation appears, shifted two to three layers deeper, so the model spends its first layers reconstructing position before it can build the local heads. As a side effect the run was 8% faster without the table. This is a single seed, so I would not read too much into the exact size of the difference.

4 Phase 3: finetuning on reasoning and looking at attention again

4.1 Building the reasoning datasets

The task is comparative reasoning: things like "A is taller than B, B is taller than C, who is the shortest?". I generated the data programmatically (`common/reasoning_gen.py`) from a lexicon per language (`telugu/reasoning/lexicon.py`, `nepali/reasoning/lexicon.py`); the engine has no Telugu or Nepali in it and the lexicons have no logic in them.

Each example starts from a hidden ground truth, either a random ordering of three or four entities or random numeric values, and the answer is computed from that. The premises are then written out through templates, with each relation randomly phrased as "A is taller than B" or "B is shorter than A" and the sentences shuffled, so the answer cannot be read off the word order. A test re-derives every label from the hidden ordering without using the generator's own answer code, which is how I know the labels are right. There are five kinds of question, in equal numbers: who is largest/smallest in a chain; who is larger given explicit numbers (15% of these are ties, answered "equal"); what is the relation between the two ends of a chain (including a branching case, A>B and A>C, where the honest answer is "cannot be determined"); yes/no questions over a chain; and writing out the full ordering. There are seven attributes (height, age, weight, running speed and exam marks for people; price and weight for objects) and 199 sentence templates per language. Two test examples, with the gold answer after the arrow:

- Telugu: శ్రీరెడ్డికి 57 ఏళ్లు. శ్రీకాంత్కి 28 ఏళ్లు. శ్రీరెడ్డి మరియు శ్రీకాంత్ లలో ఎక్కువ వయసు ఎవరిది? జవాబు: → శ్రీరెడ్డి. (Sreereddy is 57. Sreekanth is 28. Which of the two is older? Answer: Sreereddy.)
- Nepali: टिभीभन्दा झोला महँगो छ। कम्प्युटर टिभीभन्दा सस्तो छ। कम्प्युटर र झोलाको मूल्यबीच के सम्बन्ध छ? उत्तर: → कम्प्युटर झोलाभन्दा सस्तो छ. (The bag is dearer than the TV. The computer is cheaper than the TV. What is the relation between the computer's and the bag's price? Answer: the computer is cheaper than the bag.)

I tried to keep the language natural: Telugu uses the respectful plural for people, Nepali attaches its postpositions to the noun as it normally does, and Nepali numbers are in Devanagari digits. Each language has 6,000 training, 500 validation and 1,000 test examples (7,500 in total, within the 4,000–8,000 the TAs suggested). To make sure the test set measures generalisation and not memory: the entity names (540 for Telugu, 436 for Nepali, including natural compound names like వెంకటరావు and राम प्रसाद) are split into disjoint train/validation/test pools, so a test example never mentions a name the model was finetuned on; every template family keeps its last phrasing for the test set only, and half of the test examples are written entirely in those held-out phrasings; prompts are deduplicated across splits; and everything is seeded, so the script regenerates the identical files (a test checks the committed test split against the generator).

4.2 How I finetuned

The finetune script (`common/finetune.py`) starts from that language's own pretrained checkpoint, keeps the tokenizer and vocabulary exactly as they are (the answer prefix జవాబు: / उत्तर: is just text), uses a fresh AdamW, and saves checkpoints in the same resumable format as pretraining plus the position within the epoch; the bit-exact resume test from Phase 2 has a twin here. Examples are short, so I pack whole examples back to back into 512-token rows (`<s> prompt answer </s>`, about 13 per row), the same shape the model saw in pretraining. The settings I ended up with: 30 epochs, batches of 16 rows, learning rate 3×10^{-4} with 50 warmup steps and cosine decay to 3×10^{-5}, weight decay 0.1, dropout 0.1, fp16, and the loss computed on the answer tokens only. Validation exact match on the whole validation split every 50 steps chooses the checkpoint I keep. Each run took under six minutes on a T4.

I want to be open about the fact that my first attempt did not work, because fixing it taught me the most useful thing in this phase. Run 1 used 30,000 examples but only 76 names, a plain language-modelling loss over the whole sequence, and a learning rate of 10^{-4}. Its validation accuracy stopped improving at about 15% after 50 steps, and on the "who is largest" questions it scored 0.5%, which is far worse than the 33% you get by picking any name from the prompt at random. When I looked at the outputs, the model was answering with names from the *training* set that were not in the question at all. With only 50 training names, each one was the correct answer about 200 times, so the cheapest thing to learn was "answers are names like Ravi" rather than "copy the right name out of the premises". The full-sequence loss made it worse, because it also asks the model to predict the first mention of every random name inside the premises, which is only possible by guessing from a prior over names, the exact habit that is wrong at the answer position. So for run 2 I made the name pool about six times larger with compound names, trained on the answer tokens only, and raised the learning rate (10^{-4} is the rate pretraining *ended* at, and the model never left its starting basin).

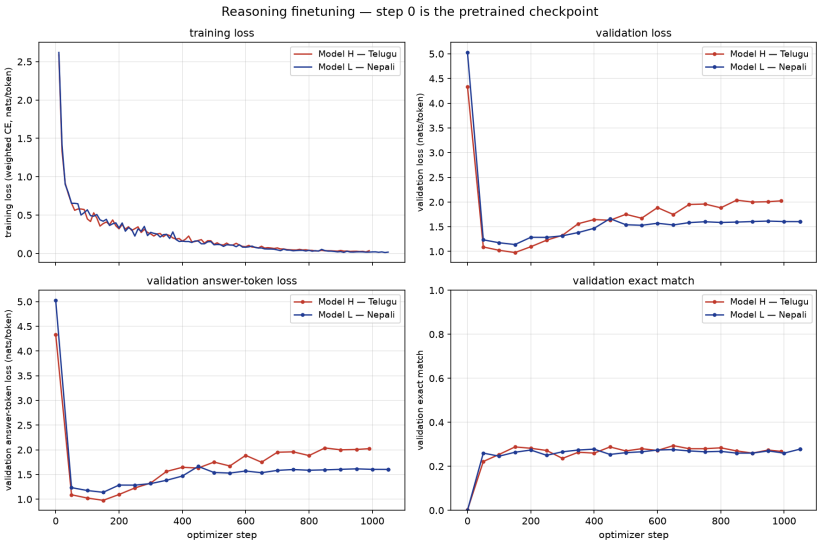


Figure 3: Finetuning curves for run 2. The pretrained model is step 0.

Two things in these curves shape everything that follows. The validation answer loss is lowest after about five epochs (step 150) and then rises again while the training loss keeps falling to almost zero: after epoch 5 the model is memorising the 6,000 training examples. But exact match does not fall, it just stays flat between 0.22 and 0.29. So whatever the model learned that generalises, it learned in the first hundred steps or so, and the rest of the training only made it more confident on training names.

4.3 Results

I report two accuracies on the 1,000 test examples. Exact match is the strict one: greedy decoding after the answer prefix, and the string has to equal the gold answer after normalisation. Scored choice gives each example's candidate answers (the names in the prompt, yes/no, the possible relation statements, the orderings) to the model and picks the one it assigns the highest probability; this separates "does the model prefer the right answer" from "can it produce it in the right format", and it is the only way to get a meaningful number out of the pretrained models, which simply continue the prompt as prose when asked to answer.

	Telugu		**Nepali**	
	pretrained	finetuned	pretrained	finetuned
exact match	0.5%	**26.9%**	0.0%	**25.0%**
answer is at least a valid option	3.2%	60.7%	0.0%	59.4%
scored choice (chance 31.9%)	28.8%	36.1%	28.3%	36.4%
largest/smallest in a chain (chance about 31%)	2.5%	31.5%	0.0%	22.0%
larger given numbers (33%)	0.0%	29.0%	0.0%	24.0%
relation between the ends (33%)	0.0%	12.5%	0.0%	16.0%
yes/no (50%)	0.0%	55.0%	0.0%	53.0%
full ordering (about 12%)	0.0%	6.5%	0.0%	10.0%
seen phrasing / held-out phrasing		28.8% / 25.2%		27.3% / 22.7%
1 hop / 2 hops / 3 hops		39.6% / 21.2% / 19.3%		35.7% / 20.9% / 14.9%
questions about people / about objects		32% / 15%		31% / 10%

Table 6: Reasoning accuracy on the test set (unseen names; half the examples also in unseen phrasings).

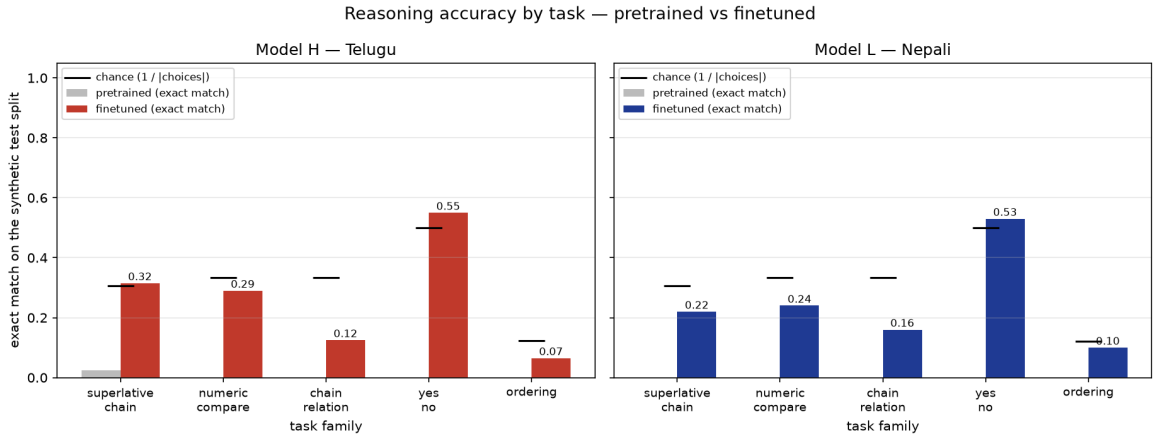


Figure 4: Exact match per question type, pretrained versus finetuned, with the chance level marked.

What I make of this: finetuning clearly worked in the sense that both models went from producing nothing usable to producing a well-formed answer 60% of the time and a correct one about a quarter of the time, and the two languages behave almost identically on every row. But when I compare each row to its chance level, the models are not really reasoning. Picking the largest in a chain is at the level of picking a random name from the prompt; yes/no is a coin flip; and accuracy halves going from one hop to three, which a model that actually chains the relations would not show. Unseen wording only costs 3–5 points, so the models are not just matching template strings. Questions about objects are much worse than questions about people, partly because the test set only has six object nouns and partly because those sentences put the entity in a different place.

Looking at the mistakes. Of the 1,000 Telugu/Nepali predictions, 269/250 are right, 338/344 are one of the valid options but the wrong one, 59/37 are a name from the prompt when a sentence or a yes/no was needed, and 333/367 are malformed (a wrong relation sentence, a repeated name). Two habits explain most of the "valid but wrong" answers. Yes/no is a default rather than a judgement: the Telugu model says కాదు ("no") 167 times out of 200, the Nepali model says हो ("yes") 132 times. And for "who is largest" the Telugu model picks the last-mentioned name 75 times, the first-mentioned 27 times and a middle one 35 times, so it learned "the answer is a name near the end" rather than the comparison. Typical failures: for the gold బకెట్ కుర్చీ కంటే బరువు ("the bucket is heavier than the chair") the model answered బల్లు, a name, the wrong kind of answer; for a Nepali relation question with a clear answer it said भन्न सकिँदैन ("cannot be determined"), an escape answer it over-uses because it is a quarter of that question type's training labels; and for the one branching case where "cannot be determined" *was* correct, it said "yes". More examples, successes as well as failures, are in `report/phase3/stats`.

What finetuning cost. I re-scored the finetuned checkpoints on the Phase 1 test corpora with the Phase 2 evaluator. Bits per byte went from 0.577 to **0.959** for Telugu and from 0.468 to **0.926** for Nepali; perplexity from 60 to 908 and from 42 to 1613. Thirty epochs of answer-only training on 0.26M tokens of templated text erased most of the language model. Run 1, with the full-sequence loss and a lower learning rate, kept much more of it (Telugu 0.665) but learned less. The two runs bracket a trade-off I did not have time to tune: a small weight on the prompt tokens, or mixing in some pretraining text, would be the next thing to try.

4.4 Did attention change?

I ran the pretrained and the finetuned checkpoints on the same 15 test prompts (three of each type, written exactly as in training) with the Phase 2 attention tools, so any difference is due to finetuning alone. I also added one measurement specific to this task: from the positions that generate the answer, how much attention lands on the entity names in the premises, and how much on the correct entity's name in particular.

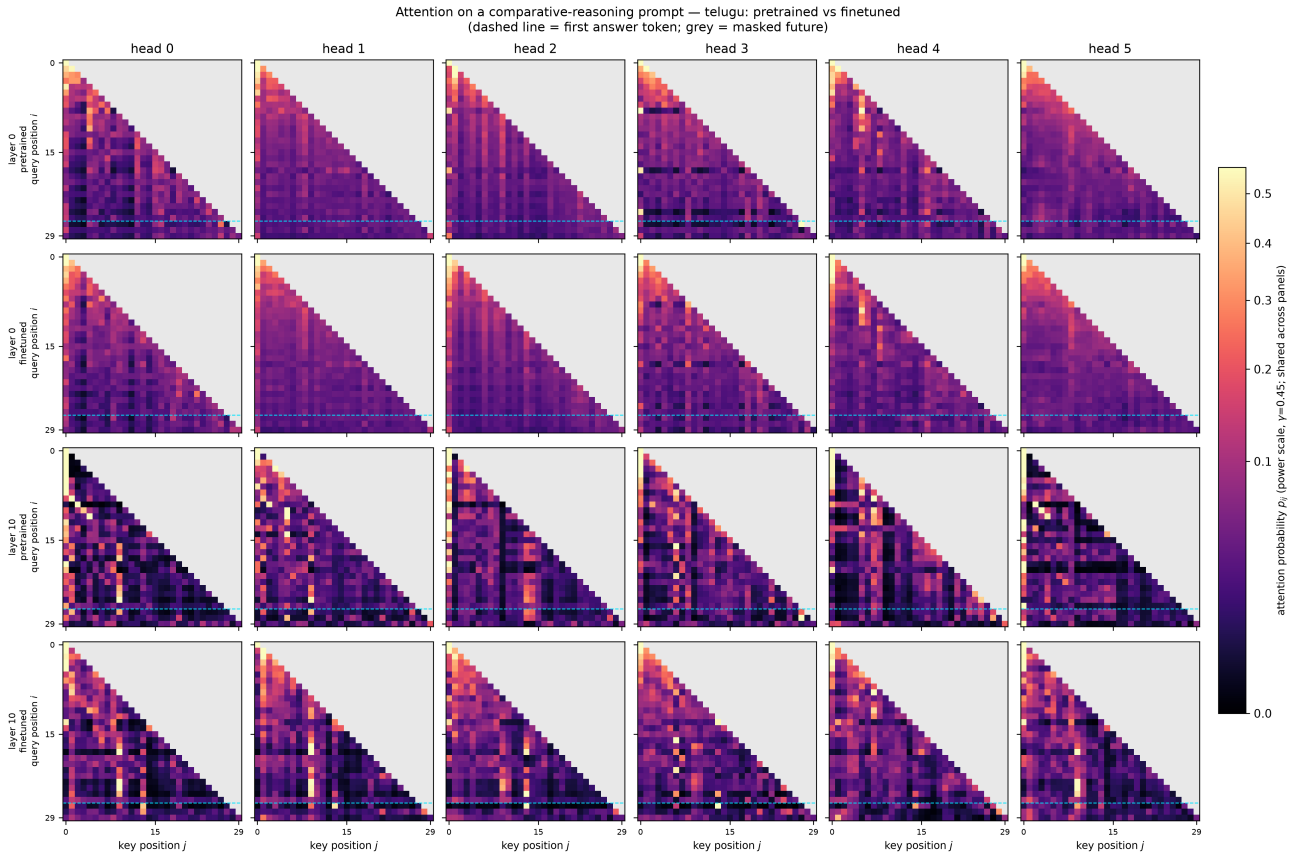


Figure 5: Model H on one test prompt: layer 0 (top two rows) and layer 10 (bottom two rows), pretrained above finetuned. The dashed line is the first answer position.

	Telugu, pretrained → finetuned	**Nepali**, pretrained → finetuned
head types (diffuse / local / content / sink)	34/15/16/1 → 46/14/6/0	27/15/22/2 → 40/12/13/1
normalised entropy, layers 6–10	0.64 → 0.73	0.62 → 0.69
look-back distance, layers 8–10 (tokens)	12.4 → 10.2	13.6 → 12.0
previous-token mass, layers 3–5	0.23 → 0.18	0.23 → 0.16
sink mass (position 0), layers 6–10	0.28 → **0.10**	0.27 → **0.12**
attention from the answer onto entity names, layers 7–10	0.22–0.37 → **0.35–0.50**	0.11–0.40 → **0.23–0.46**
share of that on the *correct* entity (best layer)	0.33 → 0.40	0.61 → 0.65
entity names as a share of all tokens (uniform baseline)	0.22	0.25

Table 7: Attention statistics on the same reasoning prompts, before and after finetuning.

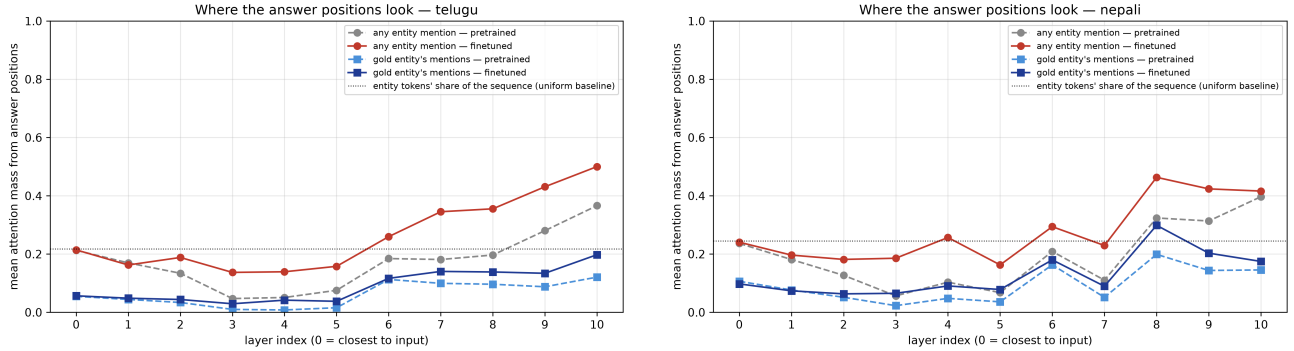


Figure 6: Attention from the answer positions onto entity names, per layer, before and after finetuning.

The early layers did not change at all. The local heads in layers 3–5 got weaker (previous-token mass down by a fifth to a quarter), which makes sense: nothing about these questions rewards predicting the next word, and the answer-only loss removed the objective that built those heads. The late layers changed the most, and in a specific way. Entropy went up and the number of content heads went down (16 to 6 in Telugu, 22 to 13 in Nepali), so heads that used to be selective became broad. The biggest single change is the sink: attention parked on position 0 in layers 6–10 fell from about 0.28 to about 0.10 in both models, and that mass reappeared on the entity names, which from

the answer position now receive 35–50% of the attention in the late layers, well above the 22–25% you would get by attending uniformly. That is the mechanism behind run 2 answering with names from the prompt instead of names from memory. But the share of that attention that lands on the *correct* entity barely moved and stays close to what a uniform spread over three names would give. So the late layers learned where the candidates are, not which relations order them, which fits the last-mentioned-name habit from the error analysis exactly. Finetuning changed the local/long-range balance and the head specialisation on these prompts, towards broader, shorter-range, name-directed attention, but it did not create a comparison circuit, and it changed both models in the same way.

5 Discussion: the four questions

How did data scale and quality differ between Model H and Model L? Telugu had 33% more training tokens available (666.5M vs 501.3M); Nepali reached the 500M target with 0.3% to spare. Since both runs used exactly 500,006,912 tokens in one pass, they saw the same amount of data during training, and Telugu's extra data would only have mattered in a longer run (Telugu saw 75% of its corpus, Nepali 99.7%). The quality differences were larger than the size difference: Telugu's manual half is mostly news, Nepali's mostly books; Nepali's downloaded half is three overlapping crawls of a small web that I could only exact-deduplicate against each other; and every Nepali document went through a Hindi/Nepali language check. The Phase 3 data was built identically for both languages.

How do the language-modelling and reasoning results compare across the two tiers? As language models, the lower-resource model won by 19% in bits per byte, with identical architecture, hyperparameters and token count, and both models ended up data-limited rather than capacity-limited, with the same attention organisation and the same decoding behaviour. On reasoning there is no gap at all: 26.9% vs 25.0% exact match, 36.1% vs 36.4% scored choice, the same per-type pattern, the same two bad habits, the same attention changes. Whatever advantage the Nepali corpus gave its model as a language model does not carry over, because the limit on reasoning is whether 25M parameters and 6,000 examples can learn a comparison procedure, and in neither language did they.

What tokenizer and corpus factors most affected the lower-resource model? The tokenizer worked in Nepali's favour: 4.34 characters per token against 3.97 means a 512-token window holds about 9% more Nepali text, an effective-context advantage that bits per byte does not cancel out, and Devanagari's smaller set of surface forms is why 16k pieces cover it more efficiently than Telugu's large akshara inventory. The corpus genre is my best explanation for the lower entropy floor: book prose is long and coherent, while news is full of names, numbers and dates that a 25M model cannot predict. The one factor against Nepali is the shared script with Hindi, which cost a language-ID pass and leaves a small contamination risk. And the corpus size is a ceiling: there is no second epoch of fresh Nepali data. For the reasoning task none of this mattered.

What evidence explains the observed differences? The language-modelling gap is a corpus effect: it survives tokenizer normalisation, the training setups are identical, and the obvious alternative explanation, OCR noise from scanned books, points the wrong way, since Nepali is *more* book-derived and still scores better. The absence of a reasoning gap is explained by what the finetuned models actually do, which I checked three independent ways: accuracy at chance on the question types that need a derivation and falling with the number of hops; the error patterns (default yes/no, last-mentioned name); and the attention measurement (late layers gather the names but put only 0.40–0.65 of that attention on the right one). The contrast between run 1 and run 2 isolates the effect of the data design (from answering with memorised names to answering with prompt names) and its price in forgetting (0.38–0.46 bits per byte), and the positional-embedding ablation isolates an architectural effect (0.041 nats, with the local heads moving two layers deeper). The main limitations: one seed per configuration, so small differences could be noise; a templated reasoning set, whose template effects the held-out-phrasing split bounds but does not remove; and a finetuning objective chosen to fix run 1's problem rather than tuned for retention.

6 Reproducibility

The README lists 15 numbered steps from crawling to the Phase 3 evaluation. Every run is seeded, every config is committed and copied into its checkpoints, 40 tests pass (model, training and resume, reasoning data and finetuning), and the Kaggle notebook that produced the Phase 3 numbers is in `notebooks/`. The corpora, tokenized binaries and all eight checkpoints (pretrained and finetuned, `best` and `latest`, for both models) are on Google Drive with open links in the README. Throughout, I used no pretrained model or tokenizer and no HuggingFace Transformer class; the two models share no data, tokenizer, vocabulary or weights and were each finetuned only from their own checkpoint; and the positional embeddings, multi-head attention and causal mask are my own implementation, with tests.